Context Diagram

The Context Diagram of LandStack – GIS-Based Integrated Land Governance Platform provides a high-level view of how the system interacts with its external entities. The LandStack system is represented at the center, while citizens, officers, administrators, and external services are connected to it through data flows. It shows the major inputs provided to the system and the information or services returned by the system.

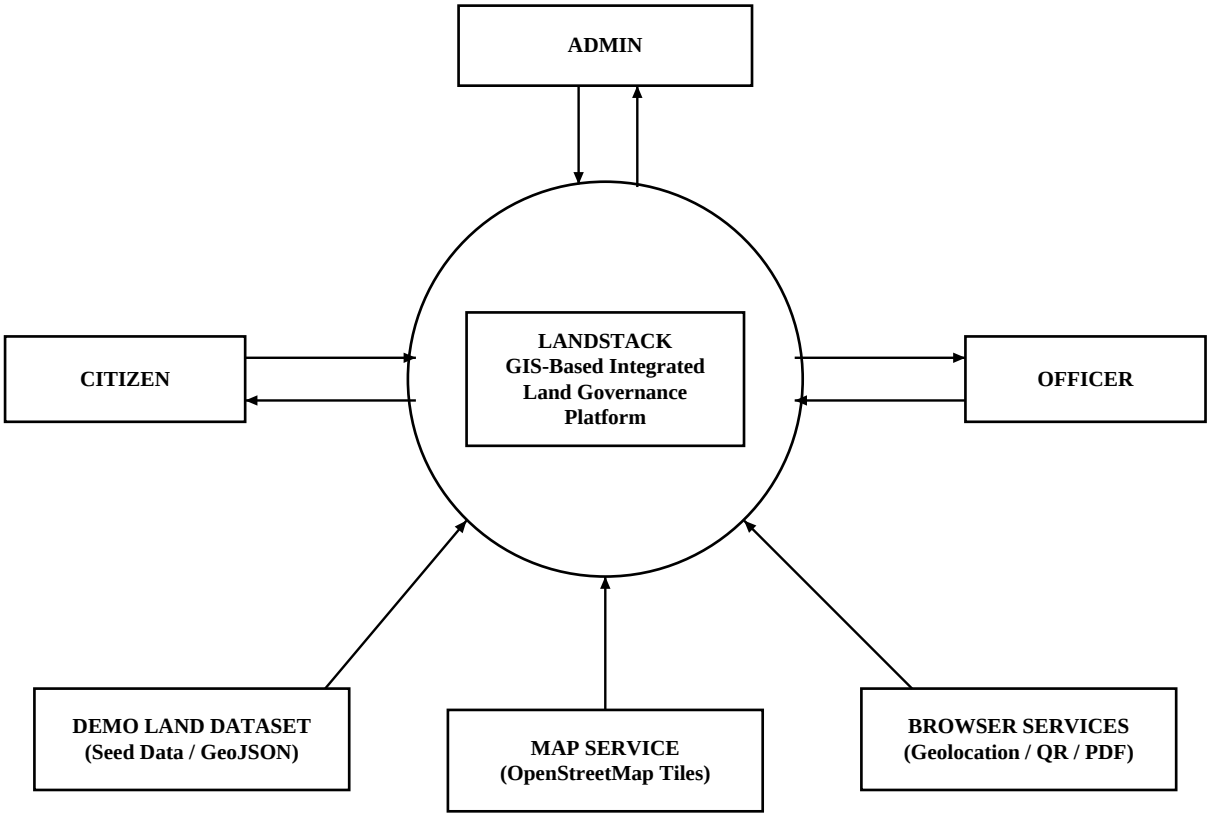

FIGURE 4.1 CONTEXT DIAGRAM

4.2 System Architecture

Overall Working of the Architecture

1. Overview of the System Architecture

The LandStack system uses a layered architecture in which each layer performs a specific function. The architecture connects citizens, officers, and administrators through a parcel-centric GIS land governance platform.

The system integrates land-related datasets around a unique parcel identifier (ULPIN) and processes parcel boundaries, ownership, RoR, registration, encumbrance, land use, restrictions, property tax, and dispute information to support land verification, map-based visualization, and potential conflict detection.

The major layers are:

- Presentation Layer
- Application / Business Logic Layer
- Intelligence & Analytics Layer
- Data Processing Layer
- Data Storage Layer
- External Services

2. Presentation Layer

The Presentation Layer provides the user interface of LandStack. It contains separate interfaces for Citizens, Officers, and Administrators.

Web Interface: Provides login, dashboards, the GIS map, parcel search, Parcel 360 pages, service requests, disputes, and reports using Next.js, TypeScript, and Tailwind CSS.

Citizen Module: Allows citizens to search parcels, view public parcel information on the map, generate verification QR codes, download land cards, submit service requests, and track their status.

Officer Module: Allows officers to view assigned service requests and disputes, verify parcel records, review potential data conflicts, and update request status.

Admin Dashboard: Allows the administrator to manage users, parcels, GIS layers, departments, service requests, disputes, potential conflicts, documents, audit logs, and system settings.

3. Application / Business Logic Layer

This layer acts as the core of the system and manages the main operations of LandStack through the Express.js REST API.

Authentication & Role-Based Access Control: Manages login, JWT-based authentication, password hashing with bcrypt, and role permissions for Citizen, Officer, and Admin. Permissions are validated on the backend for every protected API.

Parcel Management: Manages parcel records and their linked ownership, RoR, registration, encumbrance, land use, building permission, property tax, restriction, and utility information through the ULPIN.

Service Request Management: Handles request submission, request number generation, workflow status changes, and tracking.

Dispute Management: Handles the submission, assignment, and resolution of boundary, ownership, registration, land use, and tax disputes.

Document Generation: Generates QR codes for verification pages and the PDF parcel information card.

Audit Logging: Records important actions such as login, record updates, service status changes, conflict reviews, and document generation.

4. Intelligence & Analytics Layer

This layer provides GIS analysis, rule-based risk assessment, and analytics for better land governance decisions. It is designed so that a future Python machine learning service can replace or extend it.

Risk Engine: Calculates a transparent weighted risk score (0–100) from area mismatch, boundary overlap, ownership conflict, registration conflict, encumbrance, and tax issues, and explains the reasons behind the score.

Conflict Detection: Detects potential data conflicts such as duplicate survey numbers, duplicate ULPINs, ownership conflicts, registration conflicts, and multiple recent transactions. These are reported as potential conflicts, not confirmed fraud.

Boundary Overlap Detection: Uses Turf.js to find overlapping parcel polygons and calculate the overlap area.

Area Mismatch Check: Compares the GIS-calculated area with the recorded area and flags differences above a configurable threshold.

Data Analytics: Analyzes land use distribution, district-wise parcels, risk distribution, service request status, registration trends, and dispute categories for the dashboard.

5. Data Processing Layer

The Data Processing Layer prepares data for storage, GIS display, and analysis.

Data Collection: Collects synthetic parcel, ownership, registration, tax, restriction, utility, and dispute data created for the demo. No real government records are used.

Data Validation: Validates user input and records before they are stored, and handles missing, duplicate, and inconsistent data.

GeoJSON Processing: Reads and prepares synthetic parcel, land use, restriction, risk, and infrastructure layers for map display.

ULPIN Mapping: Ensures that the ULPIN is the common identifier between the database, GeoJSON, QR code, PDF card, GIS map, and APIs.

Seed Data Generation: Generates the demo dataset of parcels, users, and related records for testing and demonstration.

6. Data Storage Layer

The Data Storage Layer stores and manages the information required by the system using MySQL with Prisma ORM.

User Data: Citizen, officer, administrator, profile, and role information.

Parcel & GIS Data: Parcel details linked to GeoJSON boundaries through the ULPIN.

Land Record Data: Ownership, RoR, registration, encumbrance, mortgage, land use, building permission, property tax, restriction, and utility records.

Service Request & Dispute Data: Service requests, disputes, documents, notifications, and their status history.

Audit & Alert Data: Audit logs and potential conflict alerts used for monitoring and analytics.

7. External Services

LandStack can use external services to support its operations.

OpenStreetMap Tile Service: Provides free base map tiles displayed through Leaflet with proper attribution. No paid map API or Google Maps key is required.

Browser Geolocation API: Provides the user's current position only to temporarily center the map. The location is not stored.

Future Services: A Python machine learning service for anomaly prediction and a satellite change detection module are planned as future scope and are not implemented in the current prototype.

All data in the system is synthetic and is marked as: DEMO DATA – NOT AN OFFICIAL GOVERNMENT LAND RECORD.

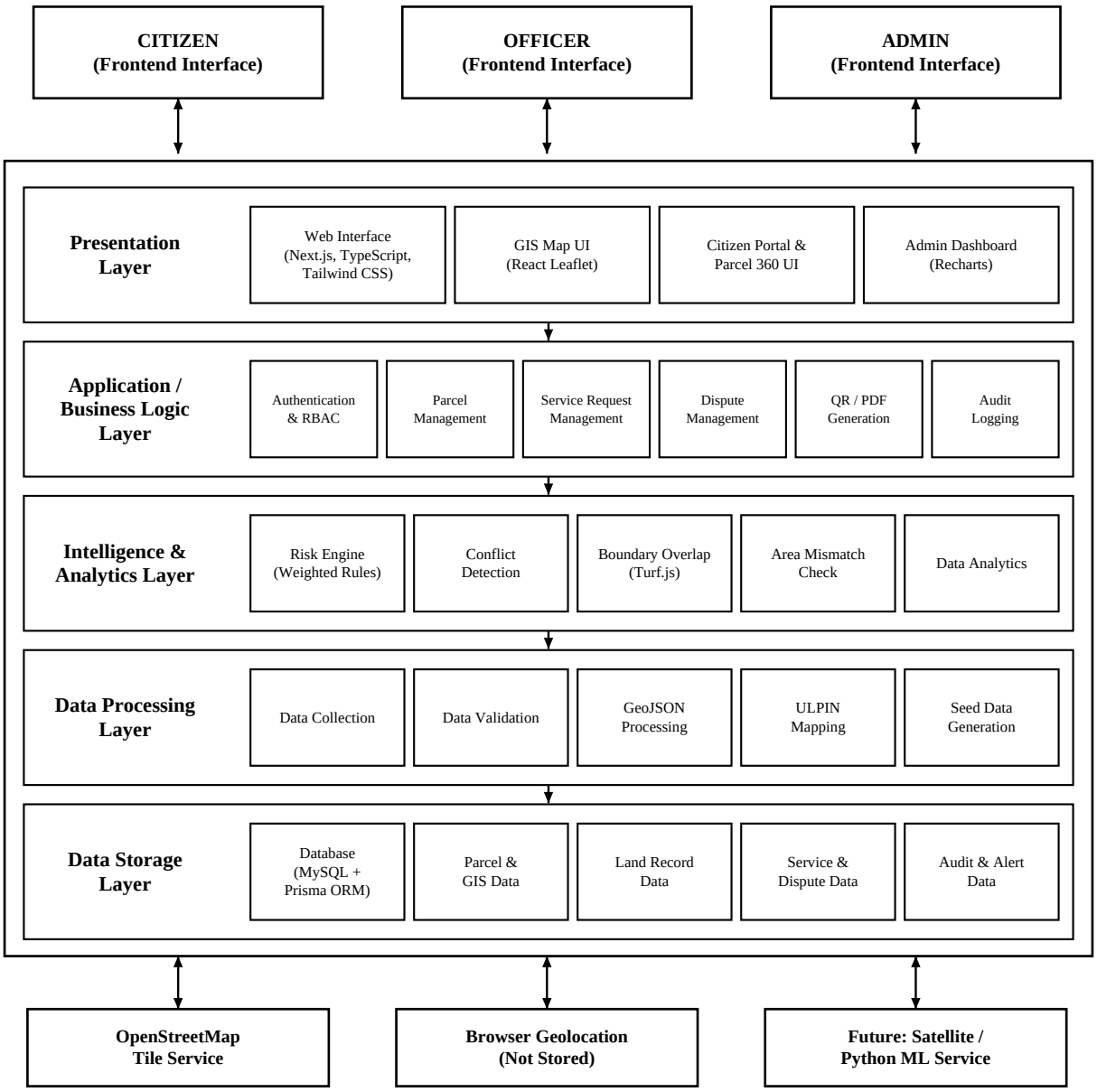

FIGURE 4.2 SYSTEM ARCHITECTURE DIAGRAM

4.3 Activity Diagram:

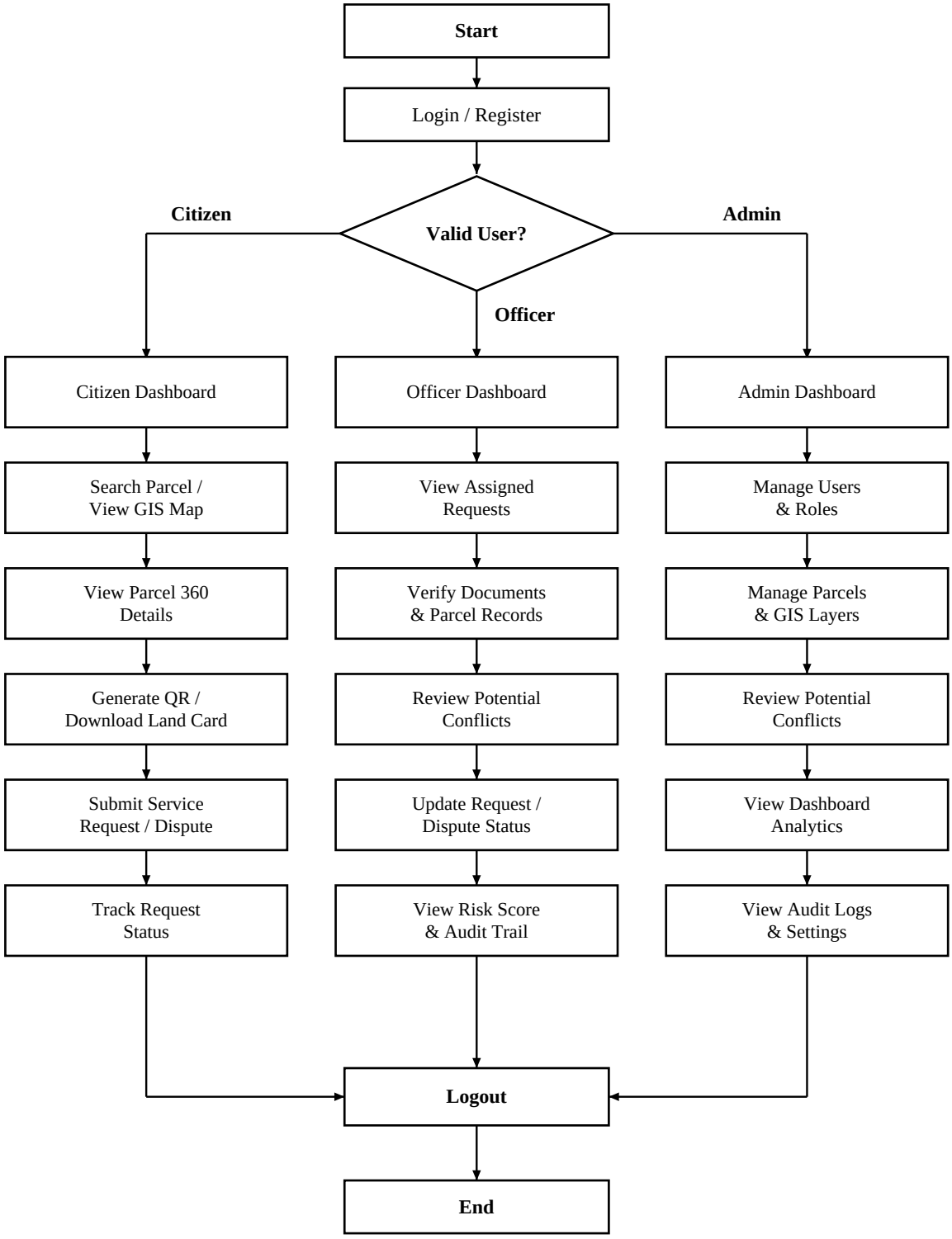

FIGURE 4.3 ACTIVITY DIAGRAM

Explanation of Activity Diagram

The Activity Diagram of LandStack represents the step-by-step workflow of the proposed system, starting from user access and ending with logout. The diagram shows how the system validates the user and directs them to the appropriate activities based on their role. The three main users are Citizen, Officer, and Admin. Each user performs different operations according to their requirements. After completing the required activities, the user logs out and the system terminates the session.

Steps of the Activity Diagram

Step 1: Start

The activity begins when the user accesses the LandStack system.

Step 2: Login / Register

The user either registers as a new citizen or logs in using existing credentials.

Step 3: Validate User

The system validates the user credentials using JWT authentication and identifies the user's role as Citizen, Officer, or Admin.

Step 4: Citizen Dashboard

If the user is a citizen, the system opens the Citizen Dashboard.

Step 5: Search Parcel and View Map

The citizen searches for a parcel by ULPIN, survey number, village, or district and views it on the GIS map. The parcel is highlighted and its popup is displayed.

Step 6: View Parcel 360

The citizen opens the Parcel 360 page to view ownership, RoR, registration, encumbrance, land use, tax, restrictions, and utilities.

Step 7: Verification and Land Card

The citizen generates the verification QR code and downloads the PDF land card.

Step 8: Service Request and Dispute

The citizen submits a service request or dispute for a parcel. The system generates a request number.

Step 9: Track Request

The citizen tracks the status of the submitted request through its workflow stages.

Step 10: Officer Dashboard

If the user is an officer, the system opens the Officer Dashboard.

Step 11: Process Assigned Records

The officer views assigned service requests and disputes, verifies documents and parcel records, and checks the risk score of the parcel.

Step 12: Review Potential Conflicts

The officer reviews potential data conflicts such as boundary overlap or area mismatch, updates their status, and adds remarks.

Step 13: Update Status

The officer updates the service request or dispute status and the citizen is notified.

Step 14: Admin Dashboard

If the user is an administrator, the system opens the Admin Dashboard.

Step 15: Manage System Data

The administrator manages users, roles, parcels, GIS layers, departments, service requests, disputes, documents, and system settings. The administrator also reviews analytics and audit logs.

Step 16: Logout and End

After completing the required activities, the user logs out and the system reaches the End state.

4.4 Use Case Diagram:

Explanation of Citizen Use Case Diagram

The Citizen Use Case Diagram shows the main functions available to the citizen in the LandStack system. The citizen interacts with the system to search parcels, view public land information on the map, verify land details, and request land-related services.

Main Use Cases

Register / Login: Allows the citizen to create an account and securely access the system.

Manage Profile: Allows the citizen to view and update personal information.

Search Parcel: Allows the citizen to search by ULPIN, survey number, village, taluk, or district.

View GIS Map: Allows the citizen to view parcels, land use, restrictions, and risk layers on the interactive map.

View Parcel 360 Details: Allows the citizen to view public information linked to a parcel.

Generate Verification QR: Allows the citizen to generate a QR code that opens the parcel verification page.

Download Land Card: Allows the citizen to download the PDF parcel information card.

Submit Service Request: Allows the citizen to request services such as ownership verification or land record information.

Track Request / Dispute: Allows the citizen to check the status of requests and disputes.

Logout: Allows the citizen to securely exit the system.

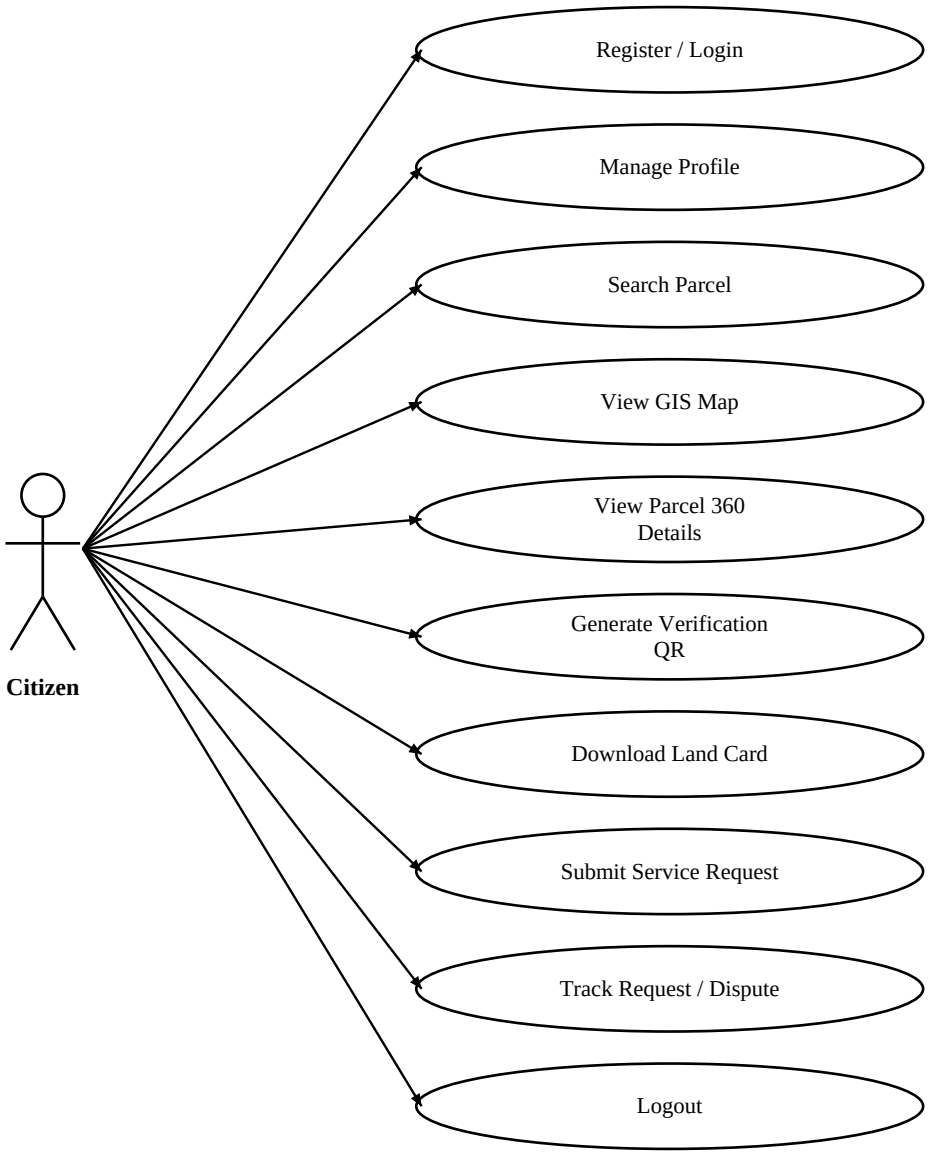

Figure 4.4.1 CITIZEN USE CASE DIAGRAM

Explanation of Officer Use Case Diagram

The Officer Use Case Diagram represents the major activities performed by an officer in the LandStack system. The officer interacts with the system to process assigned records, verify parcel data, review potential conflicts, and resolve requests and disputes.

Main Use Cases

Login: Allows the officer to securely access the officer module.

View Assigned Records: Allows the officer to view service requests and disputes assigned to them.

Verify Documents & Parcel Records: Allows the officer to check submitted documents against parcel records.

Process Service Requests: Allows the officer to move requests through review, verification, and approval or rejection.

Review Potential Conflicts: Allows the officer to review alerts such as boundary overlap and area mismatch and add remarks.

Resolve Disputes: Allows the officer to update dispute status and remarks.

View Risk Score: Allows the officer to see the risk score and the reasons behind it.

Update Parcel Records: Allows the officer to update records permitted for their role.

Logout: Allows the officer to securely exit the system.

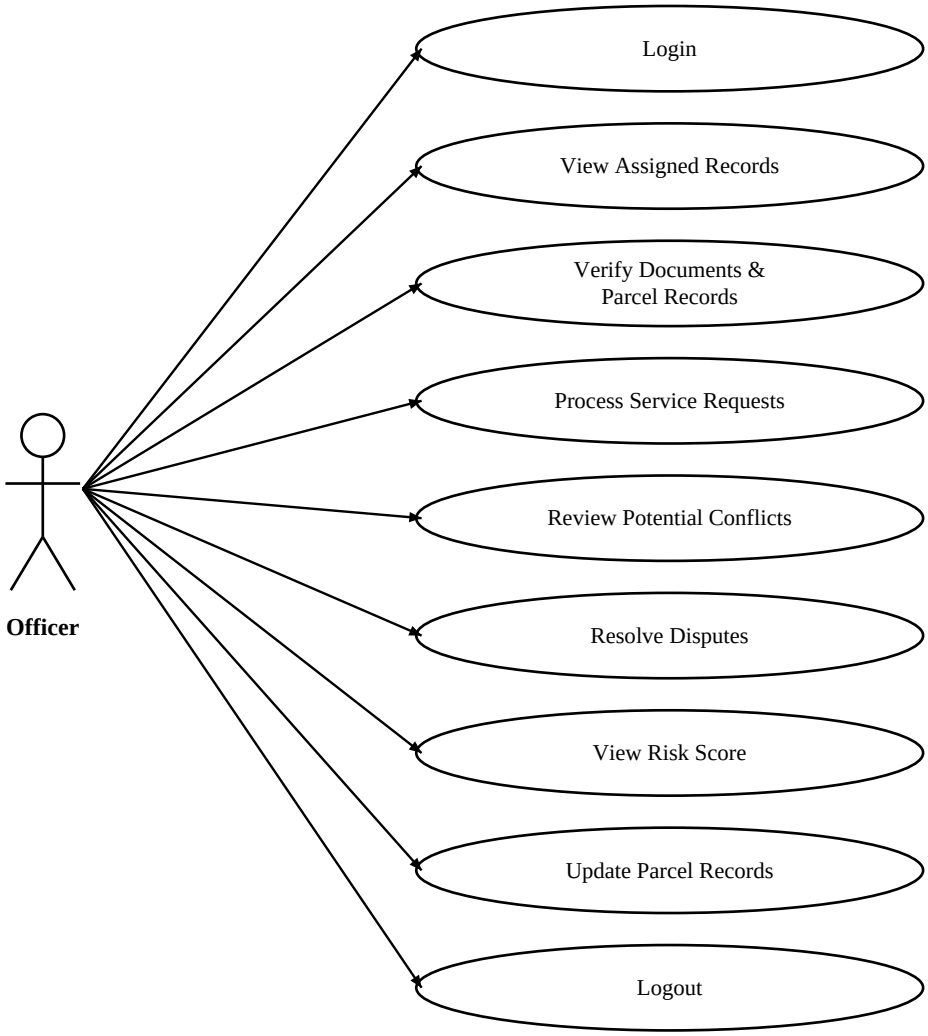

Figure 4.4.2 OFFICER USE CASE DIAGRAM

Explanation of Admin Use Case Diagram

The Admin Use Case Diagram represents the major functions performed by the administrator in the LandStack system. The administrator has control over users, parcels, GIS layers, requests, disputes, analytics, audit logs, and overall system settings.

Main Use Cases

Login: Allows the administrator to securely access the administration module.

Manage Users & Roles: Allows the admin to manage citizen, officer, and admin accounts and their roles.

Manage Parcels: Allows the admin to add, update, and monitor parcel records.

Manage GIS Layers: Allows the admin to maintain parcel, land use, restriction, risk, road, utility, and infrastructure layers.

Manage Departments: Allows the admin to maintain departments responsible for processing requests.

Monitor Requests & Disputes: Allows the admin to monitor and assign service requests and disputes.

Review Potential Conflicts: Allows the admin to review and assign potential data conflict alerts.

View Dashboard & Analytics: Allows the admin to view statistics, charts, and risk distribution.

View Audit Logs: Allows the admin to view logs of important system activities.

Manage System Settings: Allows the admin to configure settings such as the area mismatch threshold.

Logout: Allows the administrator to securely exit the system.

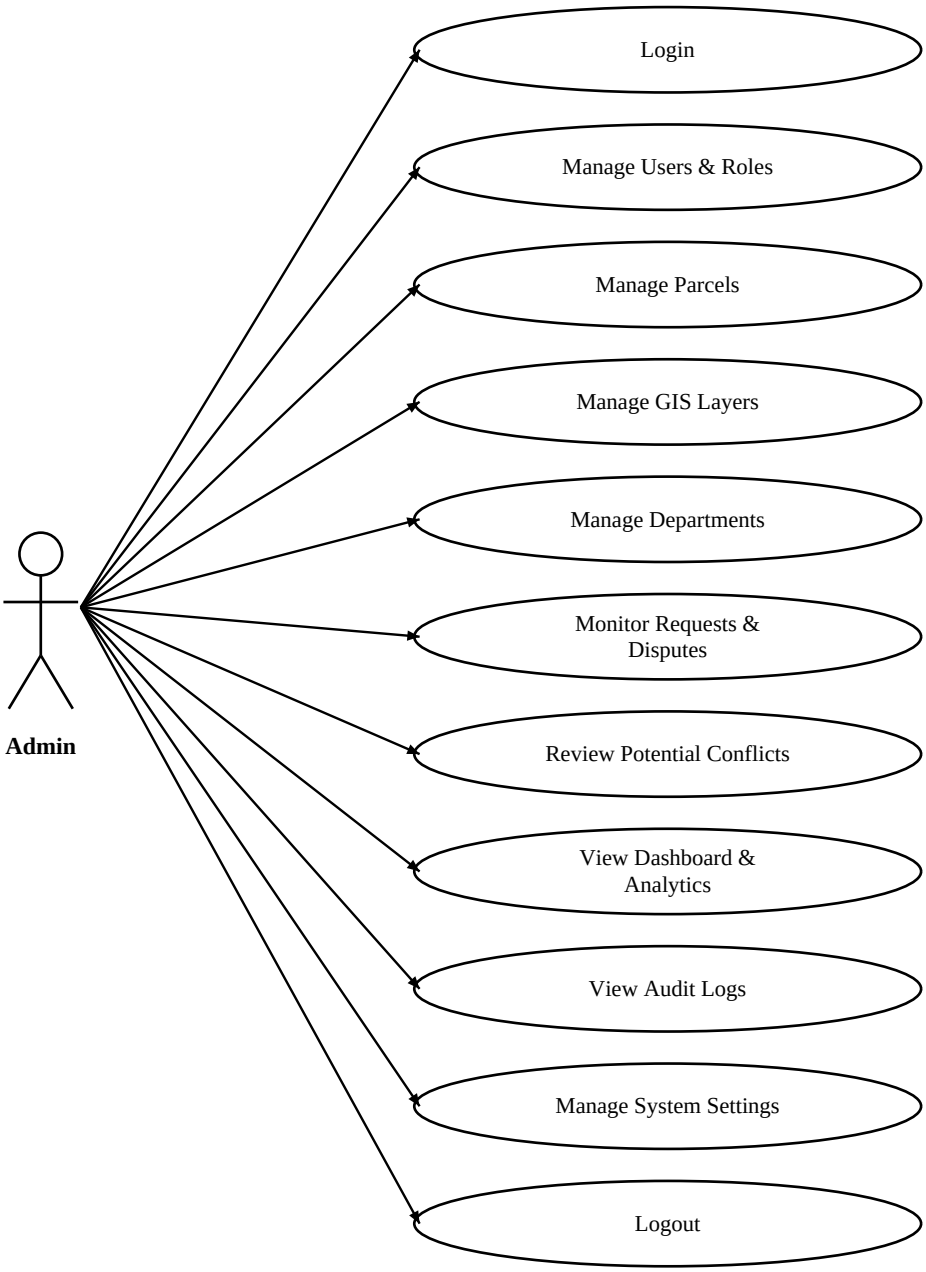

Figure 4.4.3 ADMIN USE CASE DIAGRAM

4.5 Explanation of Methodology Diagram

1. Start

The methodology begins with the identification of the project and its development process.

The initial stage establishes the overall direction of the LandStack system.

The project requirements and expected outcomes are considered before development begins.

The development process then proceeds through a sequence of defined stages.

Each stage provides the foundation for the next stage of development.

2. Requirement Analysis

This stage identifies the requirements of citizens, officers, and administrators.

The existing land record process, where land data is scattered across separate departments, and its limitations are studied.

Functional and non-functional requirements of the system are identified.

The objectives, scope, feasibility, and major system modules are defined.

The collected requirements are used as the basis for designing the system.

3. System Design (UML Diagrams)

The system structure is designed based on the identified requirements.

UML diagrams are prepared to represent the system's components and interactions.

The architecture, workflow, use cases, and activities of the system are modeled.

The Prisma database schema and the ULPIN-based parcel-centric structure are planned during this stage.

The design provides a clear blueprint for the development phase.

4. Frontend Development

The frontend development stage focuses on creating the user interface of LandStack.

Interfaces are developed for citizens, officers, and administrators using Next.js, TypeScript, Tailwind CSS, and shadcn/ui.

The GIS map, Parcel 360 pages, dashboards, service requests, and dispute pages are created using Leaflet, React Leaflet, and Recharts.

The interface is designed to be responsive on desktop, tablet, and mobile.

The frontend communicates with the backend to exchange required system information.

5. Backend Development

The backend development stage implements the main functionality of the system using Node.js, Express.js, and TypeScript.

It manages JWT authentication, role-based access control, parcels, service requests, disputes, and business logic.

The backend processes requests received from the frontend and generates appropriate responses.

It also implements the risk engine and conflict detection and handles communication between the database and application components.

Required REST APIs and system services are implemented during this stage.

6. Database Design & Integration

This stage focuses on designing and integrating the database required by LandStack.

The MySQL database stores information related to users, parcels, land records, requests, disputes, alerts, and audit logs.

Every land-related entity references the Parcel through the ULPIN.

The backend is connected to the database through Prisma ORM, and synthetic demo data is generated through seed scripts.

Proper data relationships and data management operations are implemented.

7. Testing & Debugging

Testing is performed to verify that the developed system works according to the requirements.

Individual modules such as map search, parcel click, and risk scoring are tested to identify functional errors.

The complete system is also tested to verify the interaction between different modules and roles.

Errors identified during testing are corrected through debugging.

Security, role-based access, data validation, performance, and overall functionality are also checked.

8. Deployment

After successful testing, the LandStack application is prepared for deployment.

The application is configured on the selected hosting or server environment.

Required frontend, backend, database, and environment configurations are completed.

The deployed system is checked to ensure that users can access its major functions.

Deployment makes the developed application available for demonstration and practical use.

9. Maintenance & Enhancement

After deployment, the system is continuously monitored and maintained.

Errors or issues reported by users are identified and corrected.

Demo data and GIS layers can be updated when necessary.

Existing features can be improved based on system performance and user feedback.

New functionalities such as a Python ML anomaly service and satellite change detection can be added in the future.

10. End

The methodology reaches the final stage after development, testing, deployment, and maintenance activities are completed.

At this point, the developed system has passed through the planned development process.

The final LandStack application is ready to provide its intended land governance functionality.

Further improvements can still be introduced through the maintenance and enhancement stage.

Thus, the methodology provides a structured process for developing the complete system.

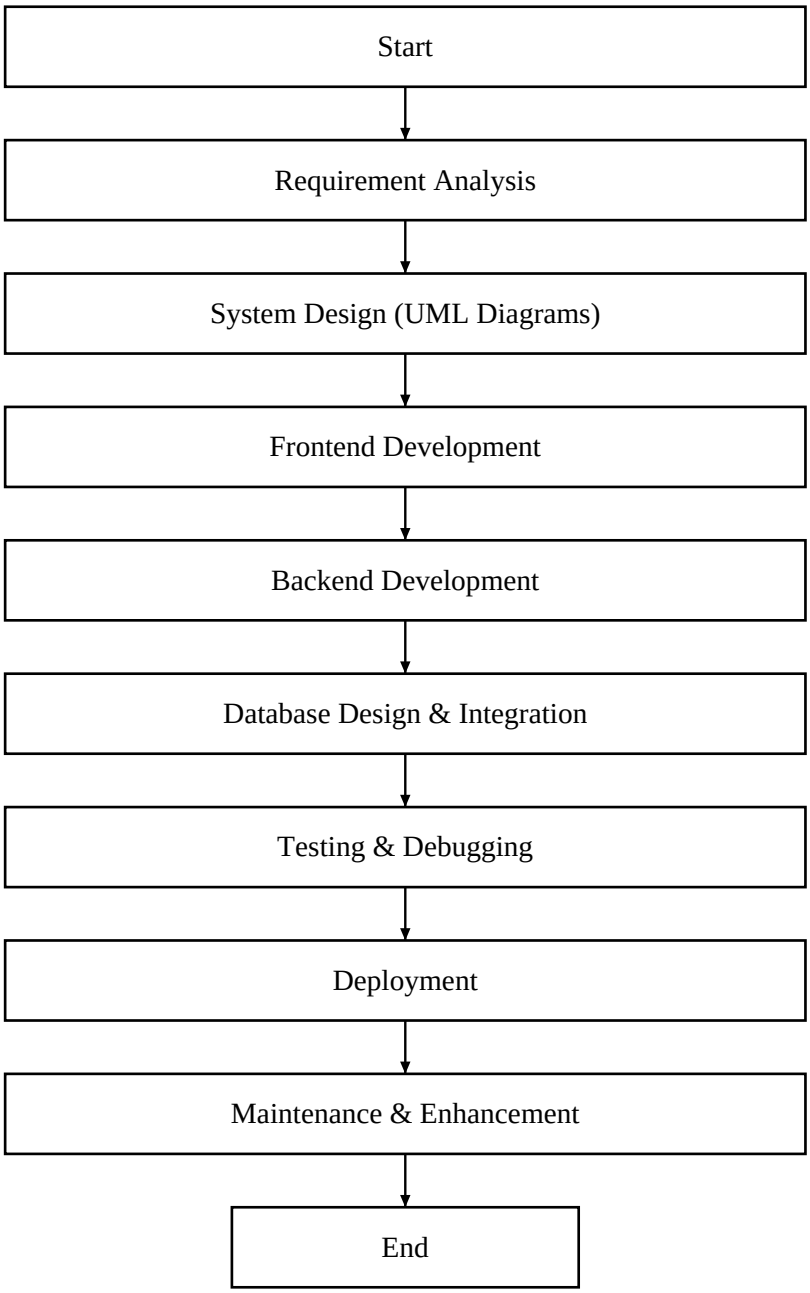

Figure 4.5.1 METHODOLOGY DIAGRAM